

Written analysis [40 points]

Submit your answers in a PDF named pa01.pdf.

Support your conclusions using your implementation, formal reasoning, and experimental results. A returned path may differ from the sample output while still being correct if it has the same optimal cost.

AI Usage Statement: I used Claude to help generate code for dc.py specifically in debugging/testing the frequency heuristic. All other code and work, including answers in the written analysis are my own. I verified Claude's work by comparing outputs against dctest.py, and manually checking the frequency heuristic's admissibility proof and work.

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—**steps**, **Scrabble**, and **frequency**—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.
2. Explain what unavoidable remaining costs it lower-bounds.
3. Explain why it cannot overestimate the true optimal remaining cost.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

Steps Heuristic

Let n be the number of character positions where the current word w differs from the goal word g .

$$h_{\text{steps}}(w) = n$$

This formula lower-bounds the absolute minimum number of singular letter edits needed to turn current word w into goal word g .

Every legal move that we take changes exactly one letter and one letter position only. If the current word w has n wrong letters, that means we have to take n letter changes to fix them all at the very least. In this method every move costs 1, and therefore the true cost of $h^*(w)$ can never be less than $n * 1 = n$. Therefore it cannot overestimate the true optimal remaining cost, and is admissible.

Scrabble Heuristic

Let n be the set of index positions where current word w and goal word g do not match. If w equals g , $h(w) = 0$. Otherwise, $h(w)$ is the sum of the Scrabble values of $g[i]$ (where i is the index) for every position i in n .

This formula lower-bounds the minimum cost required to place each needed target letter into its correct slot.

In Scrabble mode, the cost of a move is the point value of the new letter put into the word. Therefore to reach goal word g , every mismatched position i eventually will have its goal letter $g[i]$ typed in. The step that places $g[i]$ will cost at least the Scrabble value of $g[i]$. If there are more steps, those steps would only add positive costs. Eventually, adding up the Scrabble values of all target letters would give the total cost guaranteed to pay by minimum. Therefore $h_{\text{scrabble}}(w)$ never exceeds $h^*(w)$.

Frequency Heuristic

Let n be the number of mismatched letters between current word w and goal word g . Let $r(g)$ be the rarity of goal g , and r_{min} be the lowest rarity value among all dictionary words of that length.

$$\text{If } n = 0, h(w) = 0. \text{ Otherwise, } h(w) = (n - 1) \times (1 + r_{\text{min}}) + (1 + r(g)).$$

This formula lower-bounds the cost of the mandatory final move into the goal word, plus the cheapest possible cost for any of the other $n - 1$ intermediate moves.

Entering a word w costs $1 + \text{rarity}(w)$. Fixing n mismatched positions requires at least n total moves. The very last move of any solution must enter goal word g , costing exactly $1 + r(g)$. The remaining $n - 1$ moves must land on legal dictionary words, and since rarity is always ≥ 0 , no legal step can cost less than $1 + r_{\text{min}}$. Combining the exact cost of the final step with the floor cost of the other $n - 1$ steps guarantees $h_{\text{frequency}}(w)$ will never exceed the true path cost of $h^*(w)$.

Question 2: Does the heuristic actually help? [10 points]

Run both **uniform-cost search** and **A*** on:

- WHY → ASK
- BEAR → DUCK

Run each problem using all three cost measures. Report:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.3%
WHY → ASK	Frequency	38.85540652	808	38.85540652	634	21.53%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.79%
BEAR → DUCK	Frequency	38.2402284	1084	38.2402284	257	76.29%

Calculate the percentage reduction as:

$$100 \times \frac{\text{UCS expansions} - \text{A* expansions}}{\text{UCS expansions}}$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Yes, UCS and A* always returned the same optimal cost for each problem instance. This match is expected since A* is always guaranteed to find an optimal path when guided by an admissible heuristic.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

For both word pairs of WHY, ASK and BEAR, DUCK, A* produced the largest percentage reduction under the Scrabble cost measure. For WHY, ASK Scrabble had a 98.30% reduction when changing from UCS to A*. For BEAR, DUCK Scrabble had a 99.79% reduction when changing from UCS to A*.

3. For each word pair, under which cost measure was the heuristic least informative?

For both word pairs of WHY, ASK and BEAR, DUCK, frequency resulted in the lowest reduction and therefore was least informative. For WHY, ASK frequency had a 21.53% reduction from UCS to A*. For BEAR, DUCK frequency had a 76.29% reduction of expanded nodes from UCS to A*.

4. Explain the differences using the definitions of your heuristics, not runtime alone.

Out of all heuristics Scrabble performs the best. This is because in Scrabble mode, action costs depend directly on the specific letters inserted. The Scrabble heuristic sums the exact point values of the goal's target letters at mismatched positions, and because every valid path eventually types those exact letters, this heuristic gives a specific lower bound that prunes through unoptimal paths.

Next up is Steps, where every action costs the same amount of 1. Estimating the remaining cost as the number of mismatched letters n then gives a specific lowerbound that directly reflects the progress toward the goal word g .

Finally, Frequency was the least informative because an action's cost depended on the rarity of the entire word created, which was $1 + \text{rarity}(w)$. Because the rarity of the words of these in-between steps cannot be predicted in advance, the heuristic uses the dictionary's minimum rarity for all intermediate steps to guarantee admissibility. With $r_{\min}$ being very small, the formula then becomes a very loose lower bound compared to the actual rarity of the real words, and $h(w)$ significantly underestimates $h^*(w)$. The A* search then receives weaker guidance and becomes more like UCS.

Different optimal paths are acceptable when their total costs are equal.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—**steps**, **Scrabble**, and **frequency**—determine whether the optimal cost from W_1 to W_2 must always equal the optimal cost from W_2 to W_1 .

For each cost measure:

1. State **yes** or **no**.
2. Justify your conclusion.
3. If the answer is no, give a concrete counterexample using words from the provided

dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Steps Heuristic

The steps metric is symmetric. Under the steps metric, every legal action has a constant cost of 1. Therefore, because DOGCAT actions are reversible, cost symmetry holds because every edge in the search graph has the exact same weight of 1. So if an optimal path from W_1 to W_2 takes n steps, reversing that would also take n steps.

Scrabble Heuristic

The Scrabble metric is not symmetric. While actions are still reversible, Scrabble action costs are directional - therefore the cost of an action is defined by the point value of the new character inserted. So transitioning from W_1 to W_2 costs n , whereas reversing W_2 to W_1 would replace the new letter with the original character, costing m . $n \neq m$ for all possibilities so Scrabble is considered asymmetric.

Consider $W_1 = \text{cat}$ and $W_2 = \text{hat}$.

$\text{cat} \rightarrow \text{hat}$ we replace c with h and h has a value of 4.

$\text{hat} \rightarrow \text{cat}$ we replace h with c and c has a value of 3.

$4 \neq 3$, this proves that optimal costs are direction-dependent.

Frequency Heuristic

The frequency heuristic is not symmetric. Action costs under the frequency metric depend on the rarity of the resulting word w , where the formula is defined as $1 + \text{rarity}(w)$. The starting word contributes no frequency cost. Reversing a path changes which word acts as the initial, no-cost starting word. So therefore changing W_1 to W_2 would cost $1 + \text{rarity}(W_2)$ whereas changing W_2 to W_1 would cost $1 + \text{rarity}(W_1)$. Because these words all have different rarity values, the costs are asymmetric over reversible actions.

Consider $W_1 = \text{cat}$ and $W_2 = \text{hat}$.

$\text{cat} \rightarrow \text{hat}$ would cost $1 + \text{rarity}(\text{hat}) = 1 + 6.4457407 = 7.446$

$\text{hat} \rightarrow \text{cat}$ would cost $1 + \text{rarity}(\text{cat}) = 1 + 6.886906 = 7.887$

$7.446 \neq 7.887$, this proves that costs are also direction-dependent in the frequency heuristic.

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word W costs $1 + \text{rarity}(W)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(W)$ is the number of positions in which state W differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required. I use [Claude.ai](#) with model Sonnet 5 on Medium effort.

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word W costs $1 + \text{rarity}(W)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(W)$ is the number of positions in which state W differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proving admissibility and checking dominance over simple distance. >

Setup

Let W_0 be the start state, G the goal (both length- L words in the dictionary D), and let $\text{rarity} : D \rightarrow \mathbb{R}_{\geq 0}$. A move changes exactly one letter and enters the resulting word w , costing $1 + \text{rarity}(w)$. So the cost of a path $w_0 = W, w_1, \dots, w_k = G$ is

$$\text{cost} = \sum_{j=1}^k (1 + \text{rarity}(w_j)) = k + \sum_{j=1}^k \text{rarity}(w_j).$$

$h_0(W) = \text{Hamming}(W, G)$ is the usual admissible heuristic: since each move changes one position, Hamming distance to G can drop by at most 1 per move, so any path needs $k \geq h_0(W)$ moves.

Proposed heuristic

$$h_1(W) = \begin{cases} 0 & W = G \\ h_0(W) + \text{rarity}(G) & W \neq G \end{cases}$$

Idea: h_0 only lower-bounds the *number* of steps; it ignores that every one of those steps pays a rarity surcharge, and in particular that the *last* step of any path necessarily enters G itself, paying $\text{rarity}(G)$. Adding that known, unavoidable surcharge tightens the bound “for free.”

Admissibility

For $W \neq G$, any solution path has $k \geq h_0(W)$ and its last entered word is $w_k = G$. Since every term $\text{rarity}(w_j) \geq 0$,



2. A precise formula or pseudocode for the proposed heuristic.

$$\begin{aligned} h_1(W) &= h_0(W) + \text{rarity}(G) \text{ when } W \neq G \\ h_1(W) &= 0 \text{ when } W = G \end{aligned}$$

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

This heuristic is admissible, as $h_1(W) \leq \text{remaining cost}$. Any valid path to goal G takes k steps, where k will always be equal to or greater than h_0 (since h_0 is defined as the absolute minimum number of steps we must take to solve the problem). Then we can compute the total cost formula as:

$$\text{Cost} = k + \text{rarity}(w_1) + \dots + \text{rarity}(w_k) + \text{rarity}(G)$$

And because all the rarities ≥ 0 , the rarities will always be at least as large as the final rarity:

Cost $\geq k + \text{rarity}(W_1) + \dots + \text{rarity}(W_2) + \text{rarity}(G) \geq \text{rarity}(G)$

Therefore we can substitute and get the final formula of:

Cost $\geq k + \text{rarity}(G) \geq h_0(W) + \text{rarity}(G) = h_1(W)$

And since $h_1(W)$ is always less than or equal to the true remaining cost, it never overestimates and is admissible. It's also worth noting that h_1 slightly dominates h_0 when $\text{rarity}(G) \geq 0$.

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.

The heuristic that was proposed is valid, because it is admissible for any nonnegative rarity function and also dominates h_0 for any goal that has $\text{rarity}(G) > 0$. However, if $\text{rarity}(G) = 0$, h_1 becomes the same as h_0 in any case - its still admissible but less informative compared to h_0 .

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2.

Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

Testing h_0 vs h_1

Word pair	Cost measure	A* cost w/ h_0	A* nodes expanded w/ h_0	A* cost w/ h_1	A* nodes expanded w/ h_1	Reduction
WHY → ASK	Frequency	38.85540652	755	38.85540652	634	16%
BEAR → DUCK	Frequency	38.2402284	720	38.2402284	302	58.1%

h_1 reduced node expansions in both cases without changing the optimal cost, and no admissibility violation was observed in testing them; it was consistent with (though not proof of) the formal admissibility argument above, so the evidence supports adopting h_1 as at least as good as, and empirically better than, the baseline of h_0 .